

Firestore Migration Plan

Late Night Happy Hour Bowling League Site

Project: late-night-happy-hour-league-site **Database:** Firestore (NoSQL Document Database) **Date:** 2026-04-18 **Author:** Jared Bagwell

Schema Evaluation: Issues Found in Existing Design

Before defining the target schema, this section documents every bad practice, missed opportunity, and inconsistency found in the existing data structures — both in the current JSON files and in the first draft of this migration plan.

Issue 1 — Dual ID System With No Cross-Reference

Problem: Teams have local sequential numeric IDs (1–13) assigned by standings rank at transform time, AND LeaguePals MongoDB ObjectIds. The two systems are never stored together. The local ID is ephemeral — if the transform runs again with a different standings order, team IDs shift, breaking all historical foreign keys.

Fix: Store `leaguePalsId` (ObjectId string) on every `teams` document. Use the LeaguePals ObjectId as the canonical stable key for cross-referencing. Keep the local numeric `id` only as a display/sort order, not a relational key. All foreign keys in `bowlerScores`, `matchups`, and `matchupDetails` should use the LeaguePals ObjectId string.

Issue 2 — Rich Bowler Data Being Discarded

Problem: The LeaguePals raw data (`leaguepals-data/teams/{id}.json`) contains significant bowler attributes that are currently thrown away during the transform:

- `firstName`, `lastName` (parsed separately, stored only as concatenated `name`)
- `dexterity` (0 = right-handed, 1 = left-handed)
- `isFemale`, `dontIdentify` (gender identity)
- `isJunior` (age classification)
- `birthDate`
- `avatar` (profile image URL/path)
- `classification` (bowler classification string)
- `highGameHdcp`, `highSeriesHdcp` (handicap high scores — stored as scratch-only currently)
- `realAvg` / `realAvgFloat` (true float average vs. truncated integer)
- `gamesPlayed`
- `blindWeeksTotal`, `blindWeeksRow` (consecutive/total blind weeks)
- `indPointsWon` (individual match points)

Fix: Map name fields (`firstName`, `lastName`) and `avatarUrl` from LeaguePals into the `bowlers` collection. All other personal attributes (birthDate, gender, dexterity, isJunior, classification) are intentionally excluded to minimize personal information stored in the public database.

Issue 3 — Absent Game Detection Is Wrong

Problem: In the raw LeaguePals data, absent bowler games are stored as the string `"-"` in the `games` array, not as `0` or `null`. For example: `"games": ["-", "-", "-", "-"]`. The current blind detection strategy (comparing all game scores to the bowler's average) will produce false positives for bowlers who happened to bowl exactly their

average. Additionally, pre-bowl detection by comparing dates is fragile — LeaguePals has an `isMatch` boolean on each `weekGames` entry.

Fix: Use the actual `"-"` marker to detect absent/blind scores. Use `weekGames[date].isMatch` to determine whether a score was a regular match or a pre-bowl. Store `game1` , `game2` , `game3` as `null` (not 0) when the bowler was absent, and set `blinded: true` .

Issue 4 — No League Configuration Collection

Problem: League settings are hardcoded in the transform script (`0.85` handicap factor, number of games per night, etc.). If the league changes its handicap formula, the entire history would be wrong. The `league-public.json` file from LeaguePals contains rich configuration data that is completely ignored:

- `againstBlindScore` and `againstBlindScorePct` — blind score percentage rule
- `numPlayers` — bowlers per team (4)
- `numLanes` — total available lanes
- `minGamesforAvg` — minimum games to establish an average
- `previousGamesMin` — previous season min games threshold
- `weekday` and `time` — when the league bowls ("Thursday", "8:20 PM")
- `leagueType` — "Mens"
- `dues` , `entryFee` , `lineage` — financial structure
- `paymentWeeks` — 33 weeks paid
- `positionRounds` — "Every other night"
- `sanction` — USBC sanction number

Fix: Add a `leagueConfig` collection with a single document per season containing all league settings sourced from `league-public.json` . All handicap calculations reference this document so the formula is auditable.

Issue 5 — Position Rounds Not Tracked

Problem: The lane schedule has both `matches` and `splitMatches` arrays per week. Position rounds (where standings leaders play each other) are in `splitMatches` and are currently ignored entirely.

Fix: Add `positionRound: boolean` to both `matchups` and `scheduleWeeks` documents. When a week uses `splitMatches` instead of `matches` , set the flag accordingly.

Issue 6 — Team Aggregate Stats Discarded From Standings

Problem: The LeaguePals standings API returns rich team-level stats that are currently ignored:

- `average` (team average)
- `scratchPins` , `totalPins` , `bonusPins` — cumulative pin totals
- `pctWon` — win percentage string (e.g., `"48.21"`)
- `pointsWon` , `pointsLost` , `pointsTied` — granular point breakdown
- `maxGame` — team high game for the season
- `totalPinsToDate` , `gamesToDate` — running cumulative totals

Fix: Add these fields to the `teams` collection. They make standings more detailed and enable additional analytics (e.g., average differential between teams, scratch vs. handicap performance).

Issue 7 — `scheduleWeeks.dataWeek` Is Obsolete

Problem: `dataWeek` was an array index key used to look up data in the old flat JSON array files (`weeklyMatchupDetails[dataWeek]`). In Firestore, data is looked up by document ID or query — array indices do not exist. This field serves no purpose.

Fix: Remove `dataWeek` from `scheduleWeeks` . Week lookups in Firestore use `WHERE week == N` queries.

Issue 8 — `g1 / g2 / g3` Field Names Are Abbreviated

Problem: Single-character field names (`g1` , `g2` , `g3`) are cryptic in a database context. Any engineer unfamiliar with bowling will not understand them without documentation.

Fix: Rename to `game1` , `game2` , `game3` throughout all collections. `series` stays as-is — it's a standard bowling term and is unambiguous.

Issue 9 — `seasons.champion` Is a Free-Text String

Problem: `champion: "TBD"` or `champion: "Some Team Name"` cannot be reliably joined to a team record. If a team changes its name, the champion record becomes stale.

Fix: Replace with `championTeamId: string | null` (LeaguePals ObjectID FK → `teams.leaguePalsId`) and `championTeamName: string | null` (denormalized for display). Null until determined.

Issue 10 — No Audit Timestamps on Admin-Managed Collections

Problem: `announcements` , `events` , `carouselImages` , and `documents` have no `createdAt` or `updatedAt` fields. This makes it impossible to sort by creation order, audit changes, or implement "newest first" without relying on a separate `date` field with manual entry.

Fix: Add `createdAt: string` (ISO datetime) and `updatedAt: string` (ISO datetime) to all admin-managed collections. Set by the client at write time.

Issue 11 — Announcements Missing `pinned` and `expiresAt`

Problem: There is no way to pin a critical announcement to the top of the list, and no way to auto-expire time-limited announcements (e.g., "No bowling this Thursday" is irrelevant after Thursday).

Fix: Add `pinned: boolean` (pinned announcements always appear first) and `expiresAt: string | null` (ISO date after which the announcement is hidden). Front-end filters out expired announcements on read.

Issue 12 — Events Missing `endDate` and `allDay`

Problem: Multi-day events (e.g., a weekend tournament) cannot be represented. There is also no way to mark all-day events vs. time-specific events.

Fix: Add `endDate: string | null` (null for single-day events) and `allDay: boolean` .

Issue 13 — `carouselImages.image` Field Name Is Ambiguous

Problem: `image` is too generic — it could be a URL, a file path, a base64 string, or a filename. No indication of what format is expected.

Fix: Rename to `imageUrl: string` . Assume Firebase Storage URL (consistent with `documents.imageUrl`).

Issue 14 — documents Has Mutually Exclusive Fields Without Structure

Problem: `content: string | null` and `fileUrl: string | null` are both nullable, and the relationship between them is undocumented. A document could accidentally have both set or neither set.

Fix: Replace with a `source` discriminated object:

```
source: {
  type: "text" | "pdf"
  content: string | null -- populated when type == "text"
  fileUrl: string | null -- populated when type == "pdf"
}
```

Issue 15 — No Substitute Bowler Tracking

Problem: When a bowler is absent and a substitute fills in, the substitute's score is recorded under the substitute's name — but there is no record of who they were substituting for, which team they normally bowl on, or whether their score counts toward the absent bowler's stats.

Fix: Add `isSubstitute: boolean` and `substituteFor: string | null` (FK → `bowlers.id`) to `bowlerScores`.

Issue 16 — bowlerScores Conflates Match Date and Actual Bowl Date

Problem: Pre-bowlers bowl on a date different from the scheduled match. The `date` field currently stores the match date (which week it counts for). The actual date the bowler threw the ball is not recorded.

Fix: Keep `date` as the week's scheduled match date (the week the score COUNTS FOR). Add `actualBowlDate: string | null` — populated only when `preBowled == true`, otherwise null.

Issue 17 — matchups.team1Score / team2Score Field Names Are Ambiguous

Problem: It is unclear whether these scores are scratch, handicap, or total. In the current implementation they are scratch pin totals, but this is not obvious from the field name.

Fix: Rename to `team1ScratchScore` and `team2ScratchScore` for clarity.

Issue 18 — bowlers.enteringAvg Is Not Season-Scoped

Problem: `enteringAvg` is the previous season's average. But `bowlers` documents are upserted each season — if a bowler enters season 3, `enteringAvg` represents season 2's average, but there is no record of what their average was entering season 2. The field has no temporal context.

Fix: Add `enteringAvgSeason: string` field indicating which season the entering average came from (e.g., `"2024-2025"`). This makes the value interpretable in future seasons.

Issue 19 — bowlerStats.json Is Wrapped in { "data": [] } Inconsistently

Problem: All other `src/data/*.json` files are plain arrays. `bowlerStats.json` wraps its array in a `data` property, presumably mirroring the LeaguePals API response envelope. This inconsistency means the component reading it must unwrap the data differently.

Fix: This is a transform script bug. The output to Firestore will not have this issue since each bowler becomes an individual document, but the seed script must handle the `.data` wrapper when reading the source file.

Issue 20 — `matchupDetails` Stored as Sibling Collection Instead of Sub-Collection

Problem: `matchupDetails/{matchupId}` mirrors `matchups/{matchupId}` one-to-one, but as a top-level sibling collection. This works, but a Firestore sub-collection (`matchups/{matchupId}/details/summary`) would make the ownership relationship explicit and co-locate the data semantically.

Recommendation: Keep as sibling collection. Firestore sub-collections have limitations (you cannot query across sub-collections without Collection Group queries), and the current sibling approach actually performs better for the common use case of fetching all `matchupDetails` for a week in a single query. Document the 1:1 relationship explicitly instead.

Issue 21 — `teams.captain` May Not Reliably Come From `LeaguePals`

Problem: The `LeaguePals` standings data does not include a designated "captain" field. The captain field in the current data appears to be either manually entered or inferred from the first bowler listed on the team. If it comes from manual entry, it will drift.

Fix: Store `captainBowlerId: string | null` (FK → `bowlers.id`) in addition to `captainName: string` (denormalized). If `LeaguePals` does not expose a captain field, the admin panel should allow setting the captain manually with the ID reference maintained.

Final Firestore Collection Schema (DDL)

Collection: `leagueConfig` ★ NEW

One document per season storing league settings sourced from `league-public.json` . All business rules reference this document.

Document ID: `{seasonYear}` (string, e.g., `"2025-2026"`)

leagueConfig/{seasonYear}		
— <code>seasonYear:</code>	string	-- Matches <code>seasons.year</code>
— <code>leagueName:</code>	string	-- Full league name
— <code>leagueType:</code>	string	-- "Mens" "Womens" "Mixed" "Youth"
— <code>weekday:</code>	string	-- "Thursday"
— <code>startTime:</code>	string	-- "8:20 PM"
— <code>bowlingCenter:</code>	string	-- Center name
— <code>sanctionNumber:</code>	number	-- USBC sanction number
— <code>numTeams:</code>	number	-- Active teams this season
— <code>bowlersPerTeam:</code>	number	-- 4
— <code>gamesPerNight:</code>	number	-- 3
— <code>totalWeeks:</code>	number	-- 33
— <code>numLanes:</code>	number	-- Total available lanes
— <code>handicapPct:</code>	number	-- 0.85 (85%)
— <code>handicapBase:</code>	number	-- Base score for handicap calculation if applicable
— <code>blindScorePct:</code>	number	-- Fraction of average used for blind score
— <code>minGamesForAvg:</code>	number	-- Min games to establish current season avg
— <code>prevSeasonMinGames:</code>	number	-- Min games for entering average to count

```

└─ positionRoundSchedule: string -- "Every other night"
└─ dues:                  number  -- Weekly dues amount
└─ lineage:              number  -- Weekly lineage amount
└─ entryFee:             number  -- Season entry fee
└─ leaguePalsId:         string   -- LeaguePals MongoDB ObjectId for this league

```

Update frequency: Season setup (once per year, update if rules change mid-season) **Volume:** 1 per season

Collection: teams

Active teams with full standings stats sourced from LeaguePals standings API.

Document ID: {leaguePalsId} (string, LeaguePals MongoDB ObjectId — stable across seasons)

```

teams/{leaguePalsId}
└─ leaguePalsId:  string   -- MongoDB ObjectId from LeaguePals (document ID)
└─ displayId:    number   -- Sequential display rank (1-13, for standings sort)
└─ seasonYear:   string   -- FK → leagueConfig.seasonYear
└─ name:         string   -- Team name
└─ captainName:  string   -- Captain display name
└─ captainBowlerId: string|null -- FK → bowlers.leaguePalsId
└─ wins:         number   -- Season wins (decimal for ties, e.g., 33.5)
└─ losses:       number   -- Season losses
└─ ties:         number   -- Season ties
└─ points:       number   -- Season points (primary standings sort, decimal)
└─ pointsWon:    number   -- Raw points earned
└─ pointsLost:   number   -- Raw points conceded
└─ pctWon:       number   -- Win percentage (0.0-1.0, not string)
└─ average:      number   -- Team average (sum of bowler averages / numBowlers)
└─ scratchPins:  number   -- Cumulative scratch pin total
└─ totalPins:    number   -- Cumulative total pins (scratch + handicap)
└─ highGame:     number   -- Team high game this season

```

Indexes: seasonYear ASC, points DESC , seasonYear ASC, wins DESC **Update frequency:** Weekly **Volume:** ~13 per season

Collection: bowlers

One document per bowler per season. Contains identity data and cached aggregate stats (no score history — that lives in bowlerScores).

Document ID: {leaguePalsId} (string, LeaguePals MongoDB ObjectId)

```

bowlers/{leaguePalsId}
└─ leaguePalsId:  string   -- MongoDB ObjectId (document ID)
└─ seasonYear:    string   -- FK → leagueConfig.seasonYear
└─ teamId:        string   -- FK → teams.leaguePalsId
└─ teamName:      string   -- Denormalized for display
└─ firstName:     string
└─ lastName:      string
└─ name:          string   -- Computed: firstName + " " + lastName
└─ avatarUrl:     string|null -- Profile image URL (if provided by LeaguePals)

```

└─ average:	number	-- Current season truncated integer average
└─ averageFloat:	number	-- Current season true float average
└─ enteringAvg:	number	-- Previous season average (integer)
└─ enteringAvgSeason:	string	-- Which season enteringAvg came from (e.g., "2024-2025")
└─ highGame:	number	-- Season high game (scratch)
└─ highGameHdcp:	number	-- Season high game (with handicap)
└─ highSeries:	number	-- Season high 3-game series (scratch)
└─ highSeriesHdcp:	number	-- Season high 3-game series (with handicap)
└─ gamesPlayed:	number	-- Total games bowled this season
└─ blindWeeksTotal:	number	-- Total weeks bowled blind
└─ blindWeeksRow:	number	-- Current consecutive blind weeks streak
└─ indPointsWon:	number	-- Individual match points earned

Privacy note: Only name and avatar are stored from LeaguePals bowler profiles. Fields like birthDate, gender, dexterity, isJunior, and classification are intentionally excluded.

Indexes: seasonYear, teamId , seasonYear, average DESC , seasonYear, highSeries DESC **Update frequency:** Weekly **Volume:** ~55–60 per season

Collection: bowlerScores ★ FACT TABLE

One document per bowler per week. Central analytics table. All individual game data lives here.

Document ID: Auto-generated Firestore ID

bowlerScores/{autoId}		
└─ bowlerId:	string	-- FK → bowlers.leaguePalsId
└─ bowlerName:	string	-- Denormalized for display/search
└─ teamId:	string	-- FK → teams.leaguePalsId (bowler's team)
└─ teamName:	string	-- Denormalized
└─ opponentTeamId:	string	-- FK → teams.leaguePalsId
└─ opponentTeamName:	string	-- Denormalized
└─ matchupId:	string	-- FK → matchups (Firestore document ID)
└─ seasonYear:	string	-- FK → leagueConfig.seasonYear
└─ week:	number	-- Bowling week number (1–33)
└─ date:	string	-- Scheduled match date (ISO YYYY-MM-DD)
└─ actualBowlDate:	string null	-- Actual bowl date if preBowled, else null
└─ lanePair:	number	-- Lane pair (odd lane number, e.g., 1, 3, 5)
└─ game1:	number null	-- Game 1 score (null if absent/blinded)
└─ game2:	number null	-- Game 2 score (null if absent/blinded)
└─ game3:	number null	-- Game 3 score (null if absent/blinded)
└─ series:	number null	-- game1 + game2 + game3 (null if blinded)
└─ preBowled:	boolean	-- true if bowled before scheduled date
└─ blinded:	boolean	-- true if score was blind (absent, avg substituted)
└─ isSubstitute:	boolean	-- true if bowler was subbing for another bowler
└─ substituteFor:	string null	-- FK → bowlers.leaguePalsId of absent bowler

Detection rules for transform script:

- blinded = true when LeaguePals weekGames games array contains "-" values
- preBowled = true when weekGames[date].isMatch == false OR bowl date != scheduled match date

- `game1/game2/game3 = null` when blinded (store null, not 0)
- `series = null` when blinded

Indexes:

- `bowlerId ASC, seasonYear ASC, week ASC`
- `matchupId ASC`
- `teamId ASC, seasonYear ASC, week ASC`
- `seasonYear ASC, blinded ASC, series DESC`
- `seasonYear ASC, blinded ASC, game1 DESC`
- `seasonYear ASC, preBowled ASC`
- `isSubstitute ASC, seasonYear ASC`

Update frequency: Weekly **Volume:** ~55 bowlers × 33 weeks = ~1,815 per season

Collection: `matchups`

One document per scheduled matchup, covering both upcoming and completed matches.

Document ID: Auto-generated Firestore ID (stable, used as FK by `matchupDetails` and `bowlerScores`)

```
matchups/{autoId}
├─ leaguePalsMatchId:  string      -- LeaguePals _id from lane-schedule.json
├─ seasonYear:        string      -- FK → leagueConfig.seasonYear
├─ week:              number      -- Bowling week number (1-33)
├─ date:              string      -- ISO date (YYYY-MM-DD)
├─ team1Id:           string      -- FK → teams.leaguePalsId
├─ team2Id:           string      -- FK → teams.leaguePalsId
├─ team1ScratchScore: number|null  -- Team 1 total scratch pins (null if upcoming)
├─ team2ScratchScore: number|null  -- Team 2 total scratch pins (null if upcoming)
├─ positionRound:     boolean     -- true if this is a position round matchup
└─ completed:         boolean     -- true if scores are recorded
```

Indexes: `seasonYear, week` , `seasonYear, completed` , `team1Id` , `team2Id` **Update frequency:** Weekly
Volume: ~200 per season

Collection: `matchupDetails`

Team-level aggregate summary per completed matchup. Bowler scores live in `bowlerScores` . This collection caches team-level totals and handicap computations so standings and scoreboard reads are fast.

Document ID: Mirrors `matchups` Firestore document ID (1:1 relationship)

```
matchupDetails/{matchupDocId}
├─ matchupId:  string      -- FK → matchups (document ID)
├─ seasonYear: string      -- FK → leagueConfig.seasonYear
├─ week:       number
├─ date:       string      -- ISO date
├─ team1:      TeamSummary
└─ team2:      TeamSummary

TeamSummary {
  teamId:      string      -- FK → teams.leaguePalsId
```



```

teamName:      string    -- Denormalized
lane:          number    -- Lane pair (odd lane number)
teamAvg:       number    -- Sum of active bowlers' averages used for handicap calc
game1Total:    number    -- Team scratch total game 1
game2Total:    number    -- Team scratch total game 2
game3Total:    number    -- Team scratch total game 3
scratchSeries: number    -- game1Total + game2Total + game3Total
handicapPerGame: number  -- floor((opponentTeamAvg - myTeamAvg) × handicapPct)
handicapSeries: number  -- handicapPerGame × gamesPerNight
totalSeries:   number    -- scratchSeries + handicapSeries
points:        number    -- Match points earned (0-4)
}

```

Relationship note: `matchupDetails` is a sibling collection (not sub-collection) of `matchups` because Firestore Collection Group queries are required to query across sub-collections, and the current approach supports simpler weekly batch reads (`WHERE seasonYear == X AND week == N`).

Update frequency: Weekly **Volume:** ~200 per season

Collection: `scheduleWeeks`

Full season calendar. Maps calendar dates to bowling weeks and tracks skips, holidays, and position rounds.

Document ID: `{date}` (ISO date string, e.g., `"2025-09-04"`)

```

scheduleWeeks/{date}
├─ week:      number|null  -- Bowling week number (null if skipped)
├─ date:      string       -- ISO date (mirrors document ID)
├─ seasonYear: string     -- FK → leagueConfig.seasonYear
├─ status:    string       -- "completed" | "upcoming" | "skip"
├─ positionRound: boolean  -- true if this is a position round week
├─ skipReason: string|null  -- Holiday/reason name if skipped
└─ event:     string|null  -- Special event name

```

Note: `dataWeek` field from prior design has been removed — it was an array index artifact of the old JSON file system and has no meaning in Firestore.

Update frequency: Season setup (once per year) **Volume:** ~37 per season

Collection: `seasons`

Season metadata and final standings snapshot. One document per season.

Document ID: `{year}` (string, e.g., `"2025-2026"`)

```

seasons/{year}
├─ year:      string       -- Season label (matches leagueConfig.seasonYear)
├─ startDate: string       -- ISO date
├─ endDate:   string       -- ISO date
├─ championTeamId: string|null -- FK → teams.leaguePalsId (null until determined)
├─ championTeamName: string|null -- Denormalized champion name
└─ teams:     SeasonTeam[] -- Final ranked standings snapshot

```

```
SeasonTeam {
  teamId:  string  -- FK → teams.leaguePalsId
  name:    string  -- Team name (snapshot)
  wins:    number
  losses:  number
  ties:    number
  points:  number
}
```

Collection: documents

Versioned league documents. Bylaws, rules handbooks, prize fund structures — each versioned by season with one active version per type.

Document ID: Auto-generated Firestore ID

```
documents/{autoId}
├─ title:      string    -- Display title (e.g., "League Bylaws 2025-2026")
├─ type:       string    -- "bylaws" | "rules" | "prizefund" | "handbook" | "other"
├─ version:    string    -- Version label (e.g., "2025-2026", "v2.1", "Amended Oct 2025")
├─ seasonYear: string|null -- FK → seasons.year (null if not season-specific)
├─ effectiveDate: string  -- ISO date this version took effect
├─ active:     boolean   -- true = currently displayed version for type+season
├─ source: {
│   └─ type:    string    -- "text" | "pdf"
│   └─ content: string|null -- Markdown content (when source.type == "text")
│   └─ fileUrl: string|null -- Firebase Storage URL (when source.type == "pdf")
│ }
├─ createdAt:  string    -- ISO datetime
└─ updatedAt:  string    -- ISO datetime
```

Active version rule: Only one document should have `active == true` for a given `type + seasonYear` combination. When publishing a new version, the previous active document must be set to `active: false` in the same Firestore batch write.

Indexes:

- `type ASC, seasonYear ASC, active ASC`
- `type ASC, effectiveDate DESC`

Write protection: Requires Firebase Auth

Collection: announcements

Admin-managed announcements. Supports pinning and automatic expiry.

Document ID: Auto-generated Firestore ID

```
announcements/{autoId}
├─ title:      string    -- Announcement title
├─ message:    string    -- Full announcement body
├─ date:       string    -- ISO date posted (YYYY-MM-DD)
└─ type:       string    -- "reminder" | "event" | "info"
```

```
|— priority:  string      -- "low" | "normal" | "high"
|— pinned:    boolean     -- true = always shown at top regardless of priority/date
|— expiresAt: string|null  -- ISO date after which this is hidden (null = never expires)
|— createdAt: string      -- ISO datetime
|— updatedAt: string      -- ISO datetime
```

Front-end filter: WHERE expiresAt == null OR expiresAt > today **Sort order:** pinned DESC, priority DESC, date DESC

Collection: events

Admin-managed league events (tournaments, banquet, socials).

Document ID: Auto-generated Firestore ID

```
events/{autoId}
|— title:      string      -- Event name
|— date:       string      -- ISO date start (YYYY-MM-DD)
|— endDate:    string|null  -- ISO date end for multi-day events (null if single-day)
|— allDay:     boolean     -- true if no specific time
|— location:   string      -- Venue name
|— type:       string      -- "regular" | "tournament" | "social" | "banquet"
|— description: string     -- Event details
|— createdAt:  string      -- ISO datetime
|— updatedAt:  string      -- ISO datetime
```

Indexes: date ASC

Collection: carouselImages

Admin-managed hero carousel images on the home page.

Document ID: Auto-generated Firestore ID

```
carouselImages/{autoId}
|— title:      string      -- Image title
|— description: string      -- Caption text
|— imageUrl:   string      -- Firebase Storage URL (renamed from `image`)
|— alt:        string      -- Accessibility alt text
|— order:      number      -- Display sort order (ascending)
|— createdAt:  string      -- ISO datetime
|— updatedAt:  string      -- ISO datetime
```

Indexes: order ASC

Final Collection Summary Table

Collection	Type	Doc ID	Rows/Season	FK Uses
leagueConfig	Config	seasonYear	1	Referenced by all collections
teams	Dimension	leaguePalsId (OID)	13	bowlers, matchups, bowlerScores

bowlers	Dimension	leaguePalsId (OID)	55–60	bowlerScores, teams
bowlerScores	Fact	Auto	~1,815	bowlers, teams, matchups
matchups	Event	Auto	~200	teams, scheduleWeeks
matchupDetails	Aggregate	matchups doc ID	~200	matchups, teams
scheduleWeeks	Dimension	ISO date	~37	seasons, leagueConfig
seasons	Reference	seasonYear string	1+	teams
documents	Admin	Auto	< 30 total	seasons
announcements	Admin	Auto	< 50 active	none
events	Admin	Auto	< 20/season	none
carouselImages	Admin	Auto	< 20	none

Key Design Decisions Summary

Decision	Choice	Reason
Team FK	LeaguePals ObjectId string	Stable across seasons; numeric IDs shift with standings
Date format	ISO string (YYYY-MM-DD)	Easier React/JS handling than Firestore Timestamps
Absent games	null values	∅ would corrupt average calculations
bowlerScores granularity	One doc per bowler per week	Enables all analytics; Firestore charges per read not per field
matchupDetails placement	Sibling collection (not sub-collection)	Simpler weekly batch queries without Collection Group syntax
leagueConfig scope	One doc per season	Handicap rules may change; historical accuracy requires versioned settings
documents.source	Discriminated object	Prevents illegal state (both or neither content/fileName set)
blinded detection	LeaguePals "-" marker	Reliable; average-comparison approach produces false positives
series when blinded	null (not 0)	Prevents polluting aggregate queries with dummy zeroes
pctWon storage	number (0.0–1.0)	LeaguePals returns a string "48.21" — parse to float on import

Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{collection}/{document} {
      allow read: if true;
      allow write: if request.auth != null;
    }
  }
}
```

Data Pipeline: Impact on Transform Script

Scraping additions required

New Data Needed	Source	Where Used
League settings	league-public.json (already fetched)	leagueConfig collection
Bowler firstName/lastName	teams/{id}.json (already fetched)	bowlers collection
Bowler avatarUrl	teams/{id}.json (already fetched)	bowlers collection
Handicap high game/series	teams/{id}.json (already fetched)	bowlers collection
gamesPlayed, blindWeeks	teams/{id}.json (already fetched)	bowlers collection
Absent game "-" marker	teams/{id}.json weekGames	bowlerScores.blinded
isMatch flag	teams/{id}.json weekGames	bowlerScores.preBowled
Team scratchPins/totalPins	standings.json (already fetched)	teams collection
Team pctWon/average/maxGame	standings.json (already fetched)	teams collection
positionRound flag	lane-schedule.json splitMatches	matchups, scheduleWeeks
LeaguePals match _id	lane-schedule.json	matchups.leaguePalsMatchId

All required data is **already being fetched** by `scripts/fetch-league-data.js` . Only the transform script needs to be updated to map these additional fields.

Implementation Phases

Phase 1 — Firebase Foundation & Schema Validation

Tasks:

1. Publish Firestore security rules
2. Write `scripts/seed-firestore.js` — reads existing `src/data/*.json` and seeds all 12 collections using the new schema (handles `bowlerStats.json` `.data` wrapper)
3. Run seed script; verify all collections and document counts in Firebase Console
4. Validate Firestore indexes are auto-created from seed activity

Phase 2 — Rework Transform Script

Tasks:

1. Install `firebase-admin` SDK
 2. Configure service account in `.env`
 3. Add `leagueConfig` population from `league-public.json`
 4. Expand `teams` mapping to include all standings stats
 5. Expand `bowlers` mapping to include all rich bowler fields
 6. Rework `bowlerScores` population:
 - Use `"-"` marker for blind detection
 - Use `weekGames.isMatch` for pre-bowl detection
 - Store `null` for absent game values
 - Map `substituteFor` where applicable
 7. Add `positionRound` detection from `splitMatches`
 8. Remove `dataWeek` from `scheduleWeeks` output
 9. Fix team IDs to use `LeaguePals` ObjectIds as FKs throughout
 10. Write to Firestore via Admin SDK batch operations
-

Phase 3 — React Components: Read from Firestore

Tasks:

1. Create `src/hooks/useCollection.ts` and `src/hooks/useDocument.ts`
 2. Create domain-specific hooks
 3. Update all TypeScript types in `src/types/index.ts` to match new schema
 4. Replace all JSON imports in all components with Firestore hooks
 5. Update component logic for renamed fields (`g1` → `game1` , `team1Score` → `team1ScratchScore` , etc.)
 6. Handle `null` game values for blinded bowlers in display
 7. Rework `BylawsModal` to fetch from `documents` collection
 8. Add `expiresAt` filter and `pinned` sort to announcements
-

Phase 4 — Admin CRUD UI

Tasks:

1. Firebase Auth login at `/admin/login` with route guard
 2. Admin panels for all admin-managed collections
 3. Documents panel: upload PDF (Firestore Storage) or write markdown, manage active versions
 4. Announcements panel: `pinned` toggle, `expiresAt` date picker
 5. Events panel: date range picker for `endDate` , `allDay` toggle
-

Phase 5 — Cleanup & Optimization

Tasks:

1. Delete `src/data/*.json` files
 2. Create all Firestore composite indexes
 3. Add `onSnapshot` real-time listeners for standings and announcements
 4. Update README and CLAUDE.md
 5. Remove `dataWeek` field from any remaining references
-

Data Volume & Cost Estimate

Collection	Rows/Season	Avg Doc Size	Total/Season
leagueConfig	1	500 bytes	0.5 KB
teams	13	400 bytes	5 KB
bowlers	58	600 bytes	35 KB
bowlerScores	1,815	450 bytes	816 KB
matchups	200	350 bytes	70 KB
matchupDetails	200	900 bytes	180 KB
scheduleWeeks	37	200 bytes	7 KB
seasons	1	2 KB	2 KB
documents	< 10	50 KB avg	500 KB
announcements	< 20	350 bytes	7 KB
events	< 20	300 bytes	6 KB
carousellImages	< 10	250 bytes	2.5 KB
TOTAL	~2,384		~1.6 MB

Firestore Free Tier (Spark Plan): 1 GB storage / 50K reads / 20K writes per day **Estimated usage:** 0.16% of storage, < 5% of daily read/write limits **Cost: \$0.00 — permanently within free tier**

Migration Checklist

Phase 1 — Foundation

- ☐ Publish security rules
- ☐ Write and run seed script (12 collections)
- ☐ Handle `bowlerStats.json` `.data` wrapper in seed
- ☐ Verify all document counts and FKs in Firebase Console

Phase 2 — Transform Script

- ☐ Install `firebase-admin`
- ☐ Configure service account in `.env`
- ☐ Populate `leagueConfig` from `league-public.json`
- ☐ Expand `teams` with full standings stats
- ☐ Expand `bowlers` with all rich LeaguePals fields
- ☐ Correct `blinded` detection via `"-"` marker
- ☐ Correct `preBowled` detection via `isMatch` flag
- ☐ Store `null` for absent game values (not 0)

- ☐ Add `positionRound` from `splitMatches`
- ☐ Remove `dataWeek` from `scheduleWeeks`
- ☐ All FKs use `LeaguePals` `ObjectIds`
- ☐ Test full `npm run update-data` pipeline end-to-end

Phase 3 — React Reads

- ☐ Update all TypeScript types for new field names
- ☐ Create generic Firestore hooks
- ☐ Replace all JSON imports with Firestore reads
- ☐ Handle `null` game values in display components
- ☐ Rework `BylawsModal` → `documents` collection
- ☐ Announcements: filter expired, sort pinned first

Phase 4 — Admin UI

- ☐ Firebase Auth login + route guard
- ☐ Announcements CRUD (pinned, expiresAt)
- ☐ Events CRUD (endDate, allDay)
- ☐ Carousel CRUD (imageUrl, Firebase Storage upload)
- ☐ Documents: upload/edit, version management, active toggle

Phase 5 — Cleanup

- ☐ Delete `src/data/*.json`
- ☐ Zero TypeScript errors on build
- ☐ All composite indexes created
- ☐ README and CLAUDE.md updated